

GUPAO TECH

AI全链路自主决策 workflow 引擎 实战课（下）

智枢·建筑施工方案AI生成与审核系统



讲师：朱博 | 时间：2026年4月24日

课程总览

CONTENTS

第1次课：RAG与 workflow 基础

- 模块1：行业痛点与技术必要性分析
- 模块2：RAG项目全案架构设计思路
- 模块3：RAG全流程落地 + 工作流入门实践

第2次课：智能体与全项目整合

- 模块4：AI智能体开发原理与工具封装
- 模块5：复杂 workflow 核心要素与任务编排
- 模块6：全项目整合与生产级性能优化

GUPAO TECH

模块4：AI智能体开发与工具封装

在咕泡·学懂 





智能体核心架构



大脑 (大模型)

智能体的核心中枢，负责深度理解用户的复杂意图，并基于知识库进行初步的逻辑推理与决策生成。



工具集

智能体可调用的外部能力集合，如RAG文档检索、API接口调用、计算器等，用于扩展模型的边界能力。



记忆模块

负责存储并管理多轮对话的历史记录、上下文信息及关键中间结果，使智能体具备“长短期记忆”能力。



决策引擎

智能体的“调度官”，基于用户意图和当前状态，动态判断是直接生成回答，还是需要调用特定工具集进行处理。

本项目智能体核心职责

在本项目的施工规范问答场景中，智能体的核心任务是：精准判断用户的提问需求。如果是需要具体条文支撑的专业问题，则自动调度RAG工具检索施工规范文档库；如果是一般性的常识问题，则直接通过大模型生成回答。



智能体决策逻辑设计

核心决策闭环流程

用户输入 → 意图识别 → 工具选择 → 执行 → 结果反馈



意图识别

结合大模型与Prompt工程，精准判断用户需求的核心目标与关键信息。



工具选择

基于识别出的意图，决策系统调用最匹配的工具，本项目中主要指向RAG知识库。



执行阶段

触发并运行选定的工具（如RAG检索），获取外部数据或计算结果。



结果整合

将工具返回的原始结果进行格式化处理，或结合模型生成最终的自然语言答案。

本项目定制分支逻辑



场景 A：需要查阅专业规范

系统自动调用 RAG 工具进行知识库检索，获取精准规范条文后，再结合上下文生成合规方案。



场景 B：通用常识问题

无需外部检索，直接利用大模型的参数能力进行推理，快速生成自然、准确的回答。



核心设计思路拆解

01. 主体类结构定义

创建 `ConstructionAgent` 类作为智能体的核心载体，在 `\_\_init\_\_` 构造方法中统一完成所有核心组件的初始化工作。

02. 配置大模型“大脑”

集成 OpenAI 模型驱动智能体逻辑，通过 `.env` 环境变量灵活配置 API Key 与 Base URL，保证代码的安全性与可移植性。

03. 记忆与工具集封装

初始化列表存储对话记忆；将 RAG 检索函数封装为内部工具 `\_rag\_search\_tool`，赋予智能体调用外部知识库的能力。

核心代码：agent/agent_core.py

```
class ConstructionAgent:

    def __init__(self):
        # 1. 初始化大模型大脑
        self.llm = OpenAI(api_key=os.getenv("LLM_API_KEY"))
        # 2. 初始化记忆列表
        self.memory = []
        # 3. 注册 RAG 检索工具
        self.tools = {"rag_search": self._rag_search_tool }

    def _rag_search_tool(self, query):
        return rag_search(query) # 调用外部RAG能力
```

💡 关键点：将业务功能抽象为工具函数，实现智能体的模块化扩展。



智能体决策逻辑实现

核心代码实现 (agent/agent_core.py)

```
# 1. 意图识别: 判断是否调用 RAG
def _intent_recognize(self, query):
    prompt = f"分析需求: {query}, 只需回答“是”或“否”"
    return self.llm.predict(prompt).strip()

# 2. 智能体核心决策逻辑
def agent_decide(self, input_data):
    need_rag = self._intent_recognize(input_data["query"])

# 3. 动态分支: 按需调用工具
if need_rag == "是":
    ctx = self.tools["rag_search"](input_data["query"])
    self.memory.append({"used_rag": need_rag})

# 4. 记忆更新
input_data["retrieved_ctx"] = ctx or ""
return input_data
```

#

意图识别: 可控的 Prompt 工程

通过严格限定输出为“是/否”，强制 LLM 对用户问题进行二分类，精准判断是否需要查阅建筑施工规范，避免产生无效回答。



分支决策: 动态调用外部工具

基于意图识别结果执行条件分支：若判定为“是”，则自动触发 RAG 检索工具获取规范文档内容；否则跳过检索，实现智能体的自主行动。



记忆与数据流转闭环

将本次决策结果记录至 Memory 中，并将检索到的规范内容注入输入数据流，为后续的 LLM 回答生成提供关键上下文支持。



智能体调用测试

文件: agent/agent_core.py (续)

```
def run_agent(project_info, query):
    """运行智能体"""
    agent = ConstructionAgent()
    result = agent.agent_decide({
        "project_info": project_info,
        "query": query
    })
    return result

# 测试主程序
if __name__ == "__main__":

    # 测试场景1: 需要调用RAG检索规范
    print("测试1: 需要调用RAG - 钢筋焊接验收标准")
    res1 = run_agent("某住宅楼工程", "钢筋焊接的验收标准是什么? ")
    print("检索摘要: ", res1["retrieved_standard"][:120], "...")

    # 测试场景2: 直接生成, 无需调用RAG
    print("\n测试2: 不需要调用RAG - 施工方案通用格式")
    res2 = run_agent("某住宅楼工程", "生成一份施工方案的通用格式")
    print("RAG调用判定: ", res2["need_rag"])
```



智能体常见问题优化

核心问题与成因分析

⚠ 决策逻辑不稳定

大模型在关键节点表现异常，存在“该调用不调用”或“误调用”的随机现象。

🔍 核心成因诊断

- Prompt指令边界模糊，缺乏约束
- 大模型自身生成的随机性较高



方案一：精准优化 Prompt 工程

增加正反示例（Few-shot），使用确定性语言明确指令边界，引导模型做出符合预期的判断。



方案二：增加决策结果二次校验

在获取模型原始决策后，增加关键词匹配逻辑进行过滤，作为“守门人”机制确保输出合规。



方案三：降低模型生成随机性

适当调低大模型的 `Temperature` 参数，抑制其发散性，从而显著提升决策输出的稳定性。



智能体小结



核心：自主决策

智能体的核心在于根据外部输入的环境信息，结合内部预设的决策逻辑，动态地决定下一步的执行行动。



关键：工具封装

将具体的功能逻辑封装成标准化的“工具”接口，是智能体能够扩展能力边界、灵活调用外部资源的关键前提。



轻量：用好 Prompt

面对简单的决策逻辑场景，无需引入复杂的智能体框架，一个设计良好、包含上下文与指令的 Prompt 就足够实现需求。

GUPAO TECH

模块5：复杂 workflow 核心要素 与任务编排

在咕泡·学懂 





AI workflow五大核心要素



状态管理

LangGraph通过统一的State对象管理数据，确保在节点间正确传递和维护。



节点编排

通过add_node方法定义节点，通过add_edge和add_conditional_edges编排执行顺序。



路由控制

通过add_conditional_edges实现条件分支、循环等复杂逻辑。



工具集成

将外部工具（如RAG、API）封装成节点函数，无缝接入LangGraph。



可观测性

LangGraph提供内置的日志记录和可视化工具，便于调试和追溯。

生产级 AI 工作流的三大必备特性



有状态： 记忆上下文与历史交互



可容错： 故障自动恢复与重试机制



可追溯： 全流程日志留存与审计



施工方案编审标准 workflows 设计



智能决策与生成

STEP 01 • 启动流程

基于项目特征库，智能体自动决策方案框架，调用生成模型输出初稿。

STEP 02 • 方案产出

根据决策结果，自动填充施工工序、资源配置、安全措施等关键内容。



AI 智能合规审核

核心判断节点

对方案进行合规性、完整性、可行性的多维校验，输出“通过”或“驳回”结论及修改建议。

智能反馈机制

若驳回，系统自动生成精准的修改指令，用于驱动方案的重新生成。



结果闭环处理

☑ 审核通过 • 归档

方案存入知识库，自动流转至下一业务环节。

🔄 审核驳回 • 重试优化

基于反馈指令重新生成，最多自动重试 2 次。

👤 多次重试失败 • 人工介入

触发人工审核机制，由专业工程师进行修正。



AI审核节点实现

💡 核心功能与逻辑

📁 目标文件: `workflow/full_workflow.py`



环境与模型初始化

加载环境变量(LLM_KEY), 初始化OpenAI大模型实例, 配置日志系统。



构造专业审核Prompt

定义“建筑工程师”角色, 明确规范条款与方案内容的对比规则及输出格式。



执行审核与结果返回

调用LLM执行预测, 记录关键日志, 并将结构化的审核结果更新至状态数据中。

node_ai_check.py (Core Logic)

```
def node_ai_check(input_data):  
  
    # 初始化LLM模型, 加载环境变量  
    llm = OpenAI(api_key=os.getenv("LLM_API_KEY"))  
  
    # 构造专业的审核提示词  
    prompt = f\"\"\"  
    角色: 资深建筑工程师, 请严格审核合规性。  
    规范: {input_data['retrieved_standard']}  
    方案: {input_data['generated_plan']}  
    输出: [结论:通过/不通过] + [详细问题描述]  
    \"\"\"  
  
    # 调用大模型进行预测  
    check_res = llm.predict(prompt)  
  
    # 记录日志并更新状态数据  
    logger.info(f\"审核完成: {check_res[:40]}...\")  
    input_data[\"check_result\"] = check_res  
    return input_data
```



条件路由与循环重试实现

核心函数 I: 路由分发 (workflow_router)

```
def workflow_router(input_data):  
    # 根据审核结果动态决定下一节点  
    if "审核结论: 通过" in input_data["check_result"]:  
        return "node_archive" # 归档节点  
    else:  
        return "node_generate_plan" # 重试节点
```

核心函数 II: 全流程循环 (run_full_workflow)

```
while current_node and state["retry_count"] <= 2:  
    if current_node == "node_ai_check":  
        # 审核不通过时触发重试计数+1  
        current_node = workflow_router(state)  
    if current_node == "node_generate_plan":  
        state["retry_count"] += 1
```

💡 核心逻辑解析: 闭环控制与容错机制

- **状态机驱动:** 使用 `while` 循环作为主引擎, 通过 `current\_node` 变量驱动不同阶段的任务节点 (生成、审核、归档)。
- **有限重试:** 设置 `max\_retries=2` 阈值, 审核失败时自动重试并累加计数, 防止陷入死循环, 超限后终止并告警。



全流程异常处理

💡 核心任务说明

📁 目标文件: workflow/full_workflow.py

🔑 关键修改逻辑

在 `run\_full\_workflow` 函数的 `while` 循环体内, 增加 `**try-except**` 异常捕获块。

当任意节点执行抛出异常时, 系统会自动记录错误日志, 并可在此处扩展重试策略或发送监控告警。

💡 **最佳实践:** 生产环境中, 建议结合 Sentry/Prometheus 等监控系统, 将异常信息实时上报, 确保故障可观测。

📄 核心代码实现 (修改 run_full_workflow)

```
def run_full_workflow(project_info, query, max_retries=2):
    state = {"project_info": project_info, "query":
    query, "retry_count": 0}
    current_node = "node_agent_decide"
    while current_node is not None and state["retry_count"] <=
    max_retries:
        try:
            if current_node == "node_agent_decide":
                state = run_agent(**state)
                current_node = "node_generate_plan"
            # ... 其他节点逻辑保持不变 ...
        except Exception as e:
            logger.error(f"节点 {current_node} 执行失败:
            {str(e)}")
            # 在此处扩展: 重试 / 发送告警通知 / 熔断保护
            break
    return state
```



AI workflows 与智能体框架选型对比



LangGraph (课程核心)

核心定位：基于LangChain的状态图引擎，专注构建有状态的多智能体应用。

核心优势：深度集成LangChain生态 | 声明式状态图API | 强大路由分支 | 内置可视化



CrewAI - 角色化多智能体协作

核心定位：聚焦多角色协同工作，让不同AI角色共同拆解并完成复杂任务。

核心优势：Role-Goal-Tool 角色设计 | 智能体自主协作 | 顶层目标导向任务分解



Autogen (微软开源) - 对话式协作

核心定位：构建能相互对话/讨论的智能体群体，通过沟通解决复杂问题。

核心优势：灵活的Group Chat模式 | 智能体间自由辩论协商 | 兼容LangChain生态



Prefect / Airflow - 数据工程调度

核心定位：传统数据管道与ETL workflow 调度工具，非AI原生但能力强大。

核心优势：成熟稳定社区强 | 复杂依赖管理 | 专为大规模离线数据处理设计



workflow 小结



逻辑核心

条件分支与循环

这是实现业务逻辑的关键。通过构建灵活的节点跳转与逻辑判断，让系统能够自动处理多样化的复杂业务场景。



系统保障

异常处理 & 日志

生产级系统的必备要素。它们保证了系统的健壮性与可维护性，帮助我们快速定位运行时错误，记录关键操作流程。



敏捷策略

优先自定义 workflow

简单业务场景的首选方案。旨在避免过度设计，以最小的开发成本快速验证业务想法，完美适配初期的快速迭代需求。

GUPAO TECH

模块6：全项目整合与生产级 性能优化

在咕泡·学懂 





全项目一体化整合

RAG + 智能体 + LangGraph工作流 · 全链路数据流转闭环体系



用户输入

系统入口，接收核心参数
project_info(工程信息)
query(用户查询意图)



智能体决策

核心调度中枢
动态决策是否调用RAG
实时更新
retrieved_standard



工作流引擎

使用LangGraph作为核心工作
流引擎
严格控制任务流转顺序
**Generate → Check →
Route**



状态数据池

全域数据共享总线
在所有模块间实时同步
确保信息一致性与连贯性



主入口代码编写

📄 核心文件: **app.py**

```
# 导入核心依赖与配置日志
import logging
from dotenv import load_dotenv
from workflow.full_workflow import run_full_workflow

# 日志配置: 设置输出格式与级别
logging.basicConfig(level=logging.INFO, format='%(asctime)s - %(levelname)s - %(message)s')
logger = logging.getLogger(__name__)

def main():
    load_dotenv()# 加载 .env 文件中的环境变量
    proj_info = input("请输入项目基本信息: ")
    user_query = input("请输入具体的编制/审核要求: ")
    logger.info("🚀 启动施工方案AI智能编审全流程...")
    final_result = run_full_workflow(proj_info, user_query)# 执行核心逻辑
    print("\n✅ === 智能编审最终输出 ===")
    print("结论: " + final_result["check_res"])
    print("方案: " + final_result["plan_text"][:500] + "...")

if __name__ == "__main__": main()
```



系统生产级优化



RAG 检索优化

调整 Chunk Size

根据规范文档的特点，找到语义最优的切片大小，平衡召回率与相关性。

增加关键词权重

对文档标题、章节编号等关键信息赋予更高权重，提升核心内容的检索优先级。

使用重排序 (Re-ranking)

引入交叉编码器对粗排结果进行二次排序，显著提升检索结果的语义相关性。



智能体决策增强

Prompt 注入决策示例

在系统提示词中加入真实的正反决策案例，构建思维链引导模型做出更符合预期的判断。

决策结果二次校验

对大模型的输出结果进行简单的关键词过滤与格式检查，拦截明显不符合规范的异常回答。



workflow健壮性保障

增加节点超时机制

为每个执行节点设置最大超时时间，防止因外部API卡顿导致整个流程无限挂起。

非致命错误自动重试

针对网络波动等瞬时错误配置重试策略，提高复杂流程的任务完成率。

可视化执行进度展示

向用户实时反馈当前流程所处的节点状态，避免用户在等待中产生焦虑。



多场景测试



01. 钢筋工程

某住宅楼钢筋工程施工方案，重点说明钢筋连接工艺和验收标准，确保构件连接强度符合设计规范。



02. 混凝土工程

某桥梁工程混凝土浇筑方案，重点说明大体积混凝土的温控措施，有效防止因温差过大产生的裂缝问题。



03. 脚手架工程

某超高层建筑落地式脚手架施工方案，重点说明搭设要求和安全措施，保障高空作业人员的施工安全。



04. 临时用电

某施工现场临时用电方案，重点说明配电系统和安全防护，严格执行三级配电两级保护的国家标准。



项目场景拓展与复用

快速改造方案



智能客服场景

将RAG知识库替换为产品手册， workflows调整为标准问答流程，快速适配客户咨询需求。



合同审核场景

替换知识库为法律法规库，重构工作流为合规审查流程，实现合同条款的自动分析与预警。



自动化报表生成

将生成节点改造为调用业务数据API，结合图表引擎，自动生成多维度的可视化数据报表。

未来扩展方向



多智能体任务协作

构建分工明确的智能体集群，如检索Agent、生成Agent、审核Agent，通过任务链实现复杂业务闭环。



垂直领域模型微调

使用行业高质量施工方案作为训练数据进行微调，大幅提升模型在特定领域的生成准确率和专业度。



Docker 容器化部署

将系统核心组件打包为Docker镜像，消除环境差异，支持在服务器、云平台等多种环境中快速一键部署。



本地部署与数据安全

核心部署流程



环境搭建：安装 Python、MySQL 等基础运行环境



代码部署：将项目核心代码完整上传至服务器目录



数据初始化：执行 SQL 脚本，创建所需数据库与表结构



知识库导入：将规范文档向量化并导入 Chroma 向量库



系统配置：修改 .env 文件，配置密钥与连接信息

全方位安全保障



核心数据加密

对敏感业务数据与用户隐私信息进行 AES-256 高强度加密存储。



RBAC 权限管控

基于角色的访问控制，严格定义不同岗位的数据可见范围与操作权限。



定期异地备份

配置自动化脚本，每日全量备份数据库与向量库，并同步至异地节点。



全链路日志审计

记录所有关键操作日志，支持事后追溯与合规性审查。

课程收尾

COURSE WRAP-UP

💡 核心知识点复盘

- 架构：RAG + 智能体 + 工作流协同
- 模块：规范检索 / 自主决策 / 流程管控
- 项目：建筑施工方案 AI 智能编审系统

📁 课后进阶任务

- 数据层：接入 MySQL 实现方案的持久化存储与快速查询
- 安全层：开发并集成用户权限与角色管理功能
- 目标：完善系统功能闭环，达到生产可用标准

🗣️ 互动答疑环节

- 回顾课程中的技术难点，探讨实际业务场景落地的可能性。
- 欢迎大家提出疑问，共同交流进步。

THANKS

谢谢观看

在咕泡·学懂AI